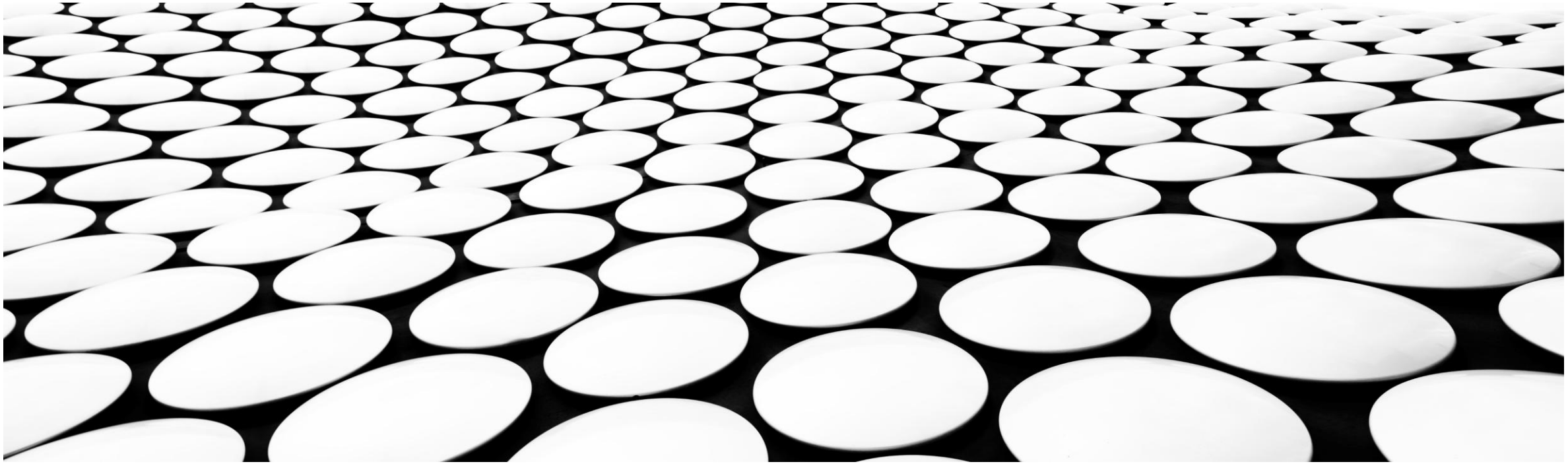
CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

TOPIC-9: **PROGRAMMING-IV**

DR. ARIJIT ROY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY PATNA



These slides are based on the book: David Money Harris and Sarah L. Harris, Computer Organization and Design

RECAP

IF STATEMENT

High-level code

```
if (i == j)
  f = g + h;

f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
```

IF STATEMENT

High-level code

```
if (i == j)
   f = g + h;

f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
bne $s3, $s4, L1
add $s0, $s1, $s2

L1: sub $s0, $s0, $s3
```

Notice that the assembly tests for the opposite case (`i != j`) than the test in the high-level code (`i == j`).

IF/ELSE STATEMENT

High-level code

```
if (i == j)
   f = g + h;
else
   f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
```

IF/ELSE STATEMENT

High-level code

```
if (i == j)
   f = g + h;
else
   f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
      bne $s3, $s4, L1
      add $s0, $s1, $s2
      j   done
L1:   sub $s0, $s0, $s3
done:
```

WHILE LOOPS

High-level code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;

while (pow != 128) {
pow = pow * 2;
x = x + 1;
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x
```

WHILE LOOPS

High-level code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;

while (pow != 128) {
  pow = pow * 2;
  x = x + 1;
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x

      addi $s0, $0, 1
      add  $s1, $0, $0
      addi $t0, $0, 128
while: beq  $s0, $t0, done
      sll  $s0, $s0, 1
      addi $s1, $s1, 1
      j    while

done:
```

Notice that the assembly tests for the opposite case (`pow == 128`) than the test in the high-level code (`pow != 128`).

FOR LOOPS

The general form of a for loop is:

FOR LOOPS

The general form of a for loop is:

```
for (initialization; condition; loop operation)  loop body
```

- `initialization:` executes before the loop begins
- `condition:` is tested at the beginning of each iteration
- `loop operation:` executes at the end of each iteration
- `loop body:` executes each time the condition is met

FOR LOOPS

High-level code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
```

FOR LOOPS

High-level code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
  sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
      addi $s1, $0, 0
      add  $s0, $0, $0
      addi $t0, $0, 10
for:  beq  $s0, $t0, done
      add  $s1, $s1, $s0
      addi $s0, $s0, 1
      j    for
done:
```

Notice that the assembly tests for the opposite case (`i == 10`) than the test in the high-level code (`i != 10`).

LESS THAN COMPARISONS

High-level code

```
// add the powers of 2 from 1
// to 100  int sum = 0;  int
i;

for (i=1; i < 101; i = i*2) {
  sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
addi $s1, $0, 0
addi $s0, $0, 1
addi $t0, $0, 101
loop:  slt $t1, $s0, $t0
       beq $t1, $0, done
       add $s1, $s1, $s0
       sll $s0, $s0, 1
       j loop
done:
```

$t1 = 1 if i < 101.

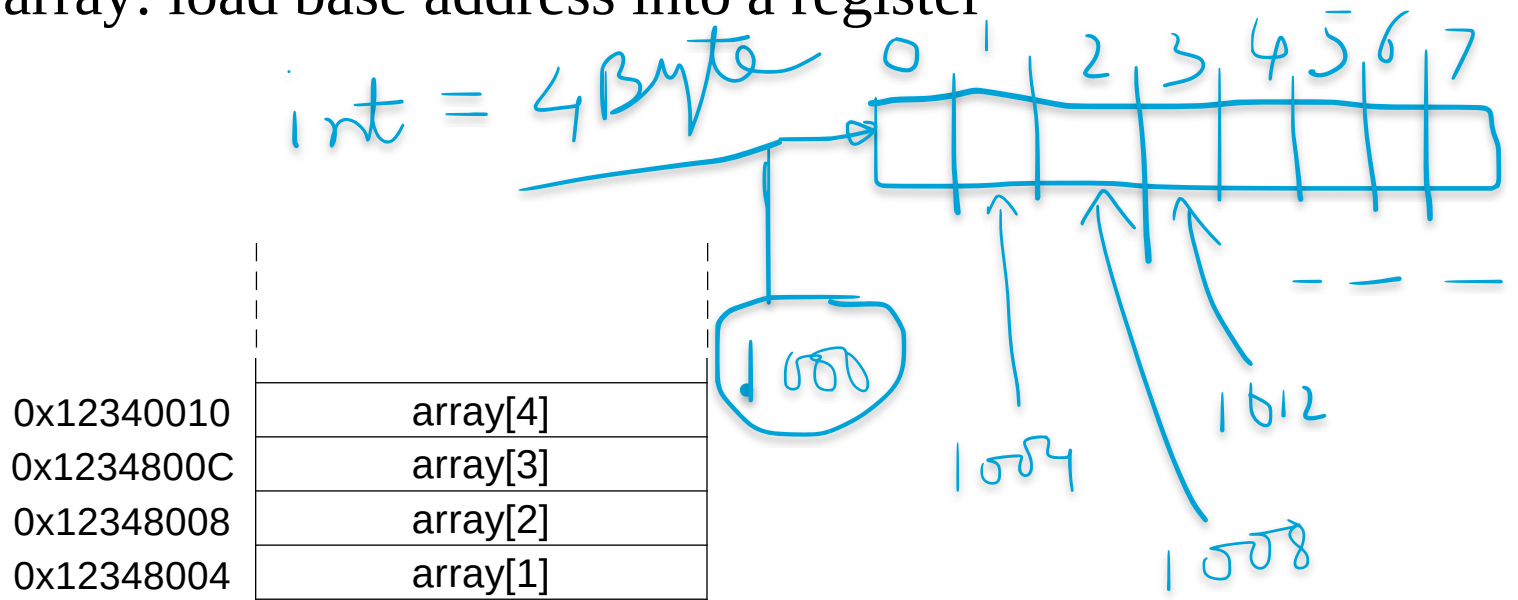
set on less than
`slt $1,$2,$3`
`if($2<$3)$1=1;`
else `$1=0`
Test if less than.
If true, set `$1` to `1`.
Otherwise, set `$1` to `0`.

ARRAYS

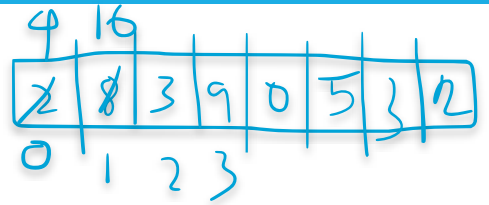
- Useful for accessing large amounts of similar data
- Array element: accessed by *index*
- Array *size:* number of elements in the array

ARRAYS

- 5-element array
- **Base address** = 0x12348000 (address of the first array element, `array[0]`)
- First step in accessing an array: load base address into a register



ARRAYS




```
// high-level code
int array[5];
array[0] = array[0] * 2;
array[1] = array[1] * 2;

# MIPS assembly code
# array base address = $s0
```

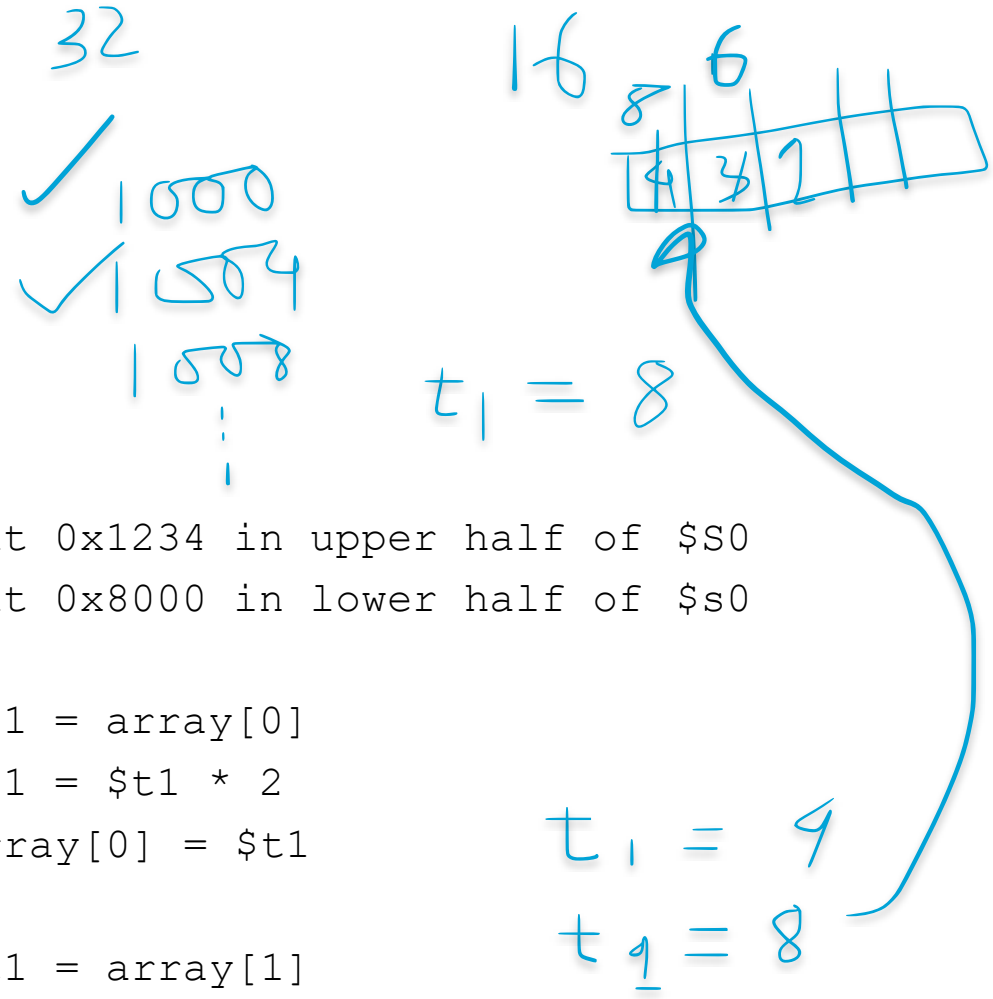
ARRAYS

```
// high-level code
   int array[5];
   array[0] = array[0] * 2;
   array[1] = array[1] * 2;

# MIPS assembly code
# array base address = $s0
   lui  $s0, 0x1234      # put 0x1234 in upper half of $S0
   ori  $s0, $s0, 0x8000 # put 0x8000 in lower half of $s0

   lw   $t1, 0($s0)      # $t1 = array[0]
   sll  $t1, $t1, 1      # $t1 = $t1 * 2
   sw   $t1, 0($s0)      # array[0] = $t1

   lw   $t1, 4($s0)      # $t1 = array[1]
   sll  $t1, $t1, 1      # $t1 = $t1 * 2
   sw   $t1, 4($s0)      # array[1] = $t1
```



ARRAYS USING FOR LOOPS


```
// high-level code
int array[1000];  int i;

for (i=0; i < 1000; i = i + 1)
array[i] = array[i] * 8;
```

ARRAYS USING FOR LOOPS

```
// high-level code
int array[1000];  int i;

for (i=0; i < 1000; i = i + 1)
array[i] = array[i] * 8;
```

set on less than
`slt $1,$2,$3`
`if($2<$3)$t=1;`
else `$t=0`
Test if less than.
If true, set `$t` to `1`.
Otherwise, set `$t` to `0`.

```
# MIPS assembly code
# $s0 = array base address, $s1  = i
# initialization code
   lui  $s0, 0x23B8          # $s0 = 0x23B80000
   ori  $s0, $s0, 0xF000     # $s0 = 0x23B8F000
   addi $s1, $0, 0           # i = 0
   addi $t2, $0, 1000        # $t2 = 1000

loop:
   slt  $t0, $s1, $t2       # i < 1000?
   beq  $t0, $0, done        # if not then done
   sll  $t0, $s1, 2          # $t0 = i * 4 (byte offset)
   add  $t0, $t0, $s0        # address of array[i]
   lw   $t1, 0($t0)          # $t1 = array[i]
   sll  $t1, $t1, 3          # $t1 = array[i] * 8
   sw   $t1, 0($t0)          # array[i] = array[i] * 8
   addi $s1, $s1, 1          # i = i + 1
   j    loop                 # repeat
done:
```

PROGRAMMING PART-IV

ASCII CODES

- *American Standard Code for Information Interchange*
  - assigns each text character a unique byte value
- For example, S = 0x53, a = 0x61, A = 0x41
- Lower-case and upper-case letters differ by 0x20 (32).

CAST OF CHARACTERS

#	Char	#	Char	#	Char	#	Char	#	Char	#	Char
20	space	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	$	34	4	44	D	54	T	64	d	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	|
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	_	6F	o		

PROCEDURE CALLS

Definitions

- Caller: calling procedure (in this case, `main`)
- Callee: called procedure (in this case, `sum`)

High-level code

```
void main()
{
  int y;
  y = sum(42, 7);
  ...
}

int sum(int a, int b)
{
  return (a + b);
}
```

PROCEDURE CALLS

Procedure calling conventions:

- Caller:
  - passes **arguments** to callee.
  - jumps to the callee
- Callee:
  - **performs the procedure**
  - **returns the result** to caller
  - **returns to the point of call**
  - **must not overwrite** registers or memory needed by the caller

MIPS conventions:

- Call procedure: jump and link (`jal`)
- Return from procedure: jump register (`jr`)
- Argument values: `$a0 – $a3`
- Return value: `$v0`

PROCEDURE CALLS

High-level code

```
int main() {
  simple();
  a = b + c;
      }

void simple() {
  return;
}
```

MIPS assembly code

```
0x00400200  main: jal simple
0x00400204        add $s0, $s1, $s2
...

0x00401020 simple: jr $ra
```

`void` means that `simple` doesn't return a value.

PROCEDURE CALLS

High-level code

```
int main() {
  simple();
  a = b + c;
      }

void simple() {
  return;
}
```

MIPS assembly code

```
0x00400200  main: jal simple
0x00400204        add $s0, $s1, $s2
...

0x00401020 simple: jr $ra
```

`jal:` jumps to `simple` and saves `PC+4` in the return address register (`$ra`).
In this case, `$ra` = 0x00400204 after `jal` executes.

`jr $ra:` jumps to address in `$ra`, in this case 0x00400204.

INPUT ARGUMENTS AND RETURN VALUES

MIPS conventions:

- Argument values: `$a0` - `$a3`
- Return value: `$v0`

INPUT ARGUMENTS AND RETURN VALUES

High-level code

```
int main()
{
   int y;
   ...
   y = diffofsums(2, 3, 4, 5);   // 4 arguments
   ...
}

int diffofsums(int f, int g, int h, int i)
{
   int result;
   result = (f + g) - (h + i);
   return result;                // return value
}
```

INPUT ARGUMENTS AND RETURN VALUES

MIPS assembly code

```
# $s0 = y

main:
   ...
   addi  $a0, $0, 2     # argument 0 = 2
   addi  $a1, $0, 3     # argument 1 = 3
   addi  $a2, $0, 4     # argument 2 = 4
   addi  $a3, $0, 5     # argument 3 = 5
   jal   diffofsums     # call procedure
   add   $s0, $v0, $0   # y = returned value
   ...

# $s0 = result
diffofsums:
   add $t0, $a0, $a1    # $t0 = f + g
   add $t1, $a2, $a3    # $t1 = h + i
   sub $s0, $t0, $t1    # result = (f + g) - (h + i)
   add $v0, $s0, $0     # put return value in $v0
   jr  $ra              # return to caller
```

INPUT ARGUMENTS AND RETURN VALUES

MIPS assembly code

```
# $s0 = result
diffofsums:
  add $t0, $a0, $a1   # $t0 = f + g
  add $t1, $a2, $a3   # $t1 = h + i
  sub $s0, $t0, $t1   # result = (f + g) - (h + i)
  add $v0, $s0, $0    # put return value in $v0
  jr  $ra             # return to caller
```

- `diffofsums` overwrote 3 registers: `$t0`, `$t1`, and `$s0`
- `diffofsums` can use the *stack* to temporarily store registers

THE STACK

- Memory used to temporarily save variables
- Like a stack of dishes, last-in-first- out (LIFO) queue
- *Expands*: uses more memory when more space is needed
- *Contracts*: uses less memory when the space is no longer needed

THE STACK

- Grows down (from higher to lower memory addresses)
- Stack pointer: `$sp`, points to top of the stack

HOW PROCEDURES USE THE STACK

- Called procedures must have no other unintended side effects.
- But `diffofsums` overwrites 3 registers: `$t0`, `$t1`, `$s0`

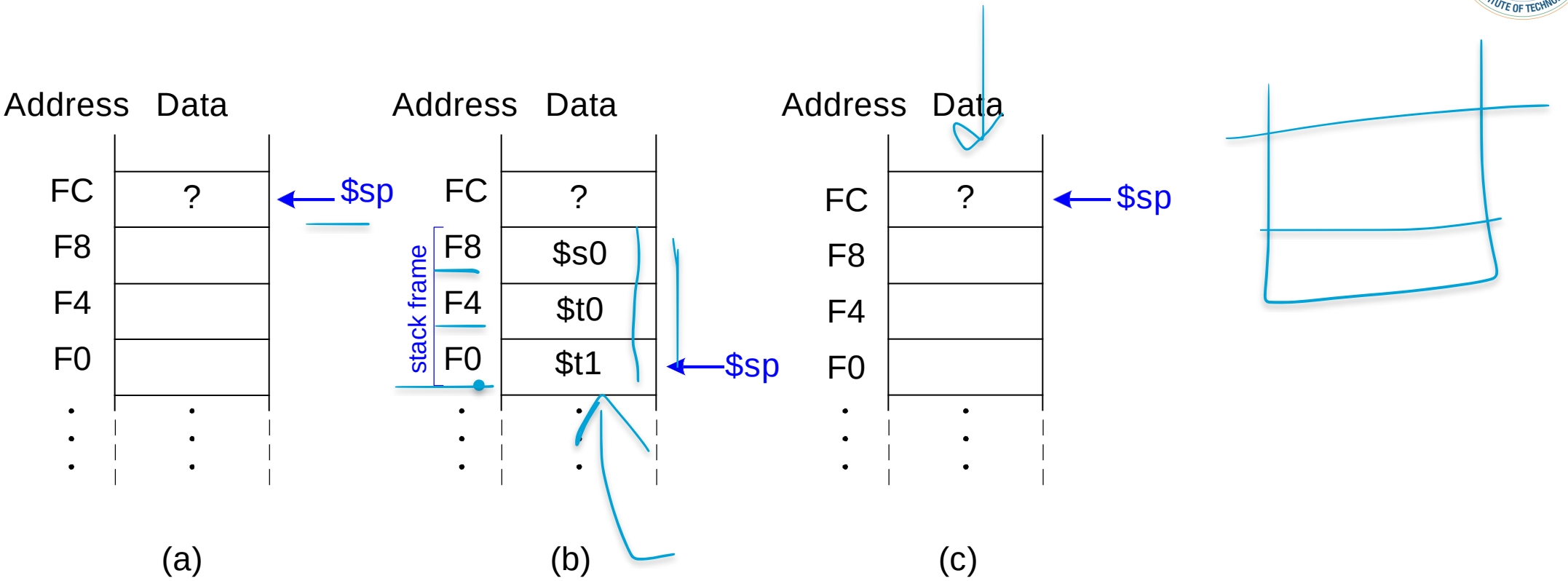
```
# MIPS assembly    # $s0 = result  diffofsums:

    add $t0, $a0, $a1   # $t0 = f + g
    add $t1, $a2, $a3   # $t1 = h + i
    sub $s0, $t0, $t1   # result = (f + g) - (h + i)
    add $v0, $s0, $0    # put return value in $v0
    jr  $ra             # return to caller
```

STORING REGISTER VALUES ON THE STACK

```
# $s0 = result
diffofsums:
   addi $sp, $sp, -12    # make space on stack
                         # to store 3 registers
   sw   $s0, 8($sp)      # save $s0 on stack
   sw   $t0, 4($sp)      # save $t0 on stack
   sw   $t1, 0($sp)      # save $t1 on stack
   add  $t0, $a0, $a1    # $t0 = f + g
   add  $t1, $a2, $a3    # $t1 = h + i
   sub  $s0, $t0, $t1    # result = (f + g) - (h + i)
   add  $v0, $s0, $0     # put return value in $v0
   lw   $t1, 0($sp)      # restore $t1 from stack
   lw   $t0, 4($sp)      # restore $t0 from stack
   lw   $s0, 8($sp)      # restore $s0 from stack
   addi $sp, $sp, 12     # deallocate stack space
   jr   $ra              # return to caller
```

THE STACK DURING DIFFOFSUMS CALL



REGISTERS

Preserved *Callee-Saved*	Nonpreserved *Caller-Saved*
`$s0 - $s7`	`$t0 - $t9`
`$ra`	`$a0 - $a3`
`$sp`	`$v0 - $v1`
stack above `$sp`	stack below `$sp`

MULTIPLE PROCEDURE CALLS

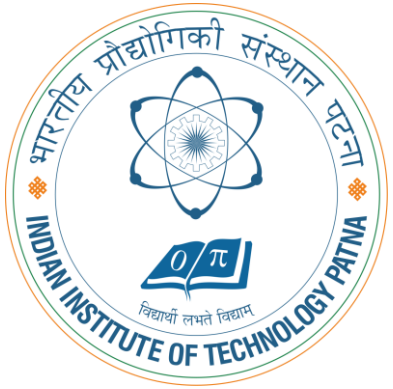
```
proc1:
    addi $sp, $sp, -4     # make space on stack
    sw   $ra, 0($sp)      # save $ra on stack
    jal  proc2
    ...
    lw   $ra, 0($sp)      # restore $s0 from stack
    addi $sp, $sp, 4      # deallocate stack space
    jr   $ra              # return to caller
```

STORING SAVED REGISTERS ON THE STACK

```
# $s0 = result
diffofsums:
addi $sp, $sp, -4      # make space on stack to
                       # store one register
sw   $s0, 0($sp)       # save $s0 on stack
                       # no need to save $t0 or $t1
  add $t0, $a0, $a1    # $t0 = f + g
  add $t1, $a2, $a3    # $t1 = h + i
  sub $s0, $t0, $t1    # result = (f + g) - (h + i)
  add $v0, $s0, $0     # put return value in $v0
  lw  $s0, 0($sp)      # restore $s0 from stack
  addi $sp, $sp, 4     # deallocate stack space
  jr  $ra              # return to caller
```

PROCEDURE CALL SUMMARY

- Caller
  - Put arguments in `$a0-$a3`
  - Save any registers that are needed (`$ra`, maybe `$t0-t9`)
  - `jal callee`
  - Restore registers
  - Look for result in `$v0`
- Callee
  - Save registers that might be disturbed (`$s0-$s7`)
  - Perform procedure
  - Put result in `$v0`
  - Restore registers
  - `jr $ra`


भारतीय प्रौद्योगिकी संस्थान पटना
INDIAN INSTITUTE OF TECHNOLOGY PATNA
विद्यार्थी लभते विद्याम्

THANK YOU!